MINISTERIO DA DEFESA
DEPARTAMENTO DE CIENCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA

ExcelCPU

Deyvisson

Sumário

1 Descrição de Objetivo **2**

2 Desenvolvimento **3**
2.1 Carregando os dados na memória 3
2.2 Chamando a função Bubble Sort 3
2.3 Construindo o Algoritmo 4
2.3.1 Implementando um Loop de Iteração 4
2.3.2 Implementando um Loop de verificação dentro do de Iteração 4
2.3.3 Implementando Comparações 5
2.3.4 Descoberta de Erro no readme.md 5

3 Conclusão do Algoritmo **6**

4 Discussão Final **7**
4.1 Como executar corretamente 7
4.2 Resultados 7

5 Referências **7**

1 Descrição de Objetivo

A partir de uma CPU desenvolvida usando planilhas Excel, criar-se-á um código Assembly destinado a essa CPU por meio de outro código Assembly destinado ao chip MIPS32. Será AssemblyExcel16 e AssemblyMIPS, respectivamente.

Segue o link do github do projeto que será utilizado na tradução, implementação e execução: **link**

Código AssemblyMIPS32 a ser traduzido:

```
# Bubble Sort Algorithm in MIPS Assembly
# Assumes that the array is stored in memory and the address of the array is in $a0
# The length of the array is in $a1

.data
array: .word 10, 21, 3, 40, 5  # Example array to sort

.text
.globl main
main:
    la $a0, array           # Load address of the array into $a0
    li $a1, 5               # Load array length into $a1
    jal bubbleSort          # Call the bubbleSort function
    li $v0, 10              # Exit program
    syscall

# Bubble Sort Function
# a0: address of the array
# a1: length of the array
bubbleSort:
    sub $a1, $a1, 1         # Decrement length for 0 indexing
    li $t0, 0               # Outer loop index i
    outer_loop:
        li $t1, 0           # Inner loop index j
        move $t2, $a0       # Pointer to the start of the array
        inner_loop:
            lw $t3, 0($t2)      # Load array[j]
            lw $t4, 4($t2)      # Load array[j+1]
            ble $t3, $t4, no_swap   # If array[j] <= array[j+1], no swap
            sw $t3, 4($t2)      # Swap elements
            sw $t4, 0($t2)
            no_swap:
            addiu $t2, $t2, 4   # Move to the next pair
            addiu $t1, $t1, 1   # Increment inner loop index
            blt $t1, $a1, inner_loop  # If j < length-1, continue inner loop
        addiu $t0, $t0, 1       # Increment outer loop index
        blt $t0, $a1, outer_loop   # If i < length-1, continue outer loop
    jr $ra                    # Return from function
```

2 Desenvolvimento

Graças ao *readme.md* disponível no github do projeto, o conjunto de instruções do AssemblyExcel16 está bem detalhado e possibilita uma tradução efetiva.

A fim de que a compreensão e desenvolvimento sejam mais fluidos, é possível a visualização de outros códigos exemplos presentes na pasta *sample programs* dentro do repositório.

2.1 Carregando os dados na memória

A seguir, as linhas iniciais do código AssemblyMIPS32:

```
.data
array: .word 10, 21, 3, 40, 5  # Example array to sort

.text
.globl main
main:
    la $a0, array          # Load address of the array into $a0
    li $a1, 5              # Load array length into $a1
```

Observa-se a declaração de uma lista de números e posterior declaração de variáveis para utilização da lista. Pode-se analisar os códigos exemplos *cycle.s* e *bouncingBall.s* a fim de traduzir como:

```
.DATA
    ; Define the list numbers, just a few so it runs quickly.
    NUM1 = $0005
    NUM2 = $0007
    NUM3 = $0009
    ; ----------

.CODE
    ; Initialize in the registers and then store in ROM.
    LOAD R0 NUM1
    STORE R0 @0000
    LOAD R0 NUM2
    STORE R0 @0001
    LOAD R0 NUM3
    STORE R0 @0002
    LOAD R0 @0000 ; Pointer to array
    LOAD R1 $0003 ; Fixed list length
```

2.2 Chamando a função Bubble Sort

Não há uma instrução para a chamada de funções, forçando que o respectivo código seja **inline** ao fluxo de execução.

Portanto, assim como é feito nos arquivos exemplos, construir-se-á dois labels que indicarão os códigos que estão sendo executados, ITERATION_LOOP e COMPARATION_LOOP e, naturalmente, os labels de comparações SWAP_CASE e NO_SWAP_CASE.

2.3 Construindo o Algoritmo

2.3.1 Implementando um Loop de Iteração

Após uma busca mais profunda sobre como ponteiros funcionam em AssemblyExcel16, pôde-se perceber, lendo o *readme.md* que a instrução LOAD é utilizada de 3 formas diferentes, uma delas sendo o acesso direto ao valor armazenado no endereço de memória. Sendo assim, não será necessário a declaração de um ponteiro.

Atentando-se a forma com que as condições são tratadas em AssemblyExcel16, pode-se construir um loop simples a fim de verificar tanto o funcionamento da instrução **LOAD** quanto do **JMP**.

```
    [...]
    DEC R1 ; Decrement so that we can iterate over the array elements.

BubbleSort:
    LOAD R2 $0000 ; Use R2 for index iteration loop

    ITERATION_LOOP:
        ; "LOAD REG REG -> loads memory unit at the address stored in REGB into REGA"
        ; --------
        LOAD R8 R2
        TRAN R1 R3
        SUB R3 R2
        JGE BubbleSort ; In case the subtraction above is equal to zero
        ; --------
        INC R2 ; In case don't.
        JMP ITERATION_LOOP
```

O código acima utiliza R3 como uma variável temporária e é capaz de iterar sobre todos os elementos do vetor, repetidamente.

2.3.2 Implementando um Loop de verificação dentro do de Iteração

Como passo intermediário, pode-se implementar a verificação de dois elementos adjacentes dois a dois, repetidamente.

```
[...]
ITERATION_LOOP:
    LOAD R3 $0000 ; Use R3 for index comparation loop
    COMPARATION_LOOP:
        ; Check two adjacent elements
        LOAD R8 R3
        INC R3
        LOAD R9 R3
        ; Verify if is finished
        TRAN R1 R10
        SUB R10 R3
        JGE ITERATION_LOOP
        JMP COMPARATION_LOOP
```

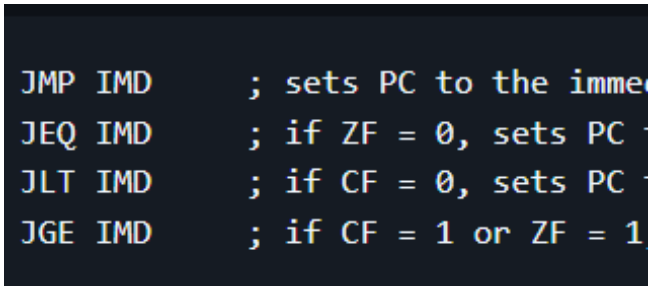
2.3.3 Implementando Comparações

Utilizando-se de labels, pode-se implementar a lógica de if-else. A planilha de instruções disponível no repositório possui uma descrição assídua de como as condicionais podem ser usadas.

```
[...]
        ; Check two adjacent elements
        LOAD R4 R3
        INC R3
        LOAD R5 R3
        ; Transform into R6 and R7 temporally for subsequent subtraction
        TRAN R4 R6
        TRAN R5 R7
        SUB R7 R6 ; Note the difference. [j+1] - [j]
        JGE NO_SWAP_CASE
            ; Load for display purposes only
            SWAP_CASE:
                LOAD R13 $0100
            NO_SWAP_CASE:
                LOAD R14 $0100
```

2.3.4 Descoberta de Erro no readme.md

Durante a implementação do uso das condicionais, percebeu-se um erro que representou um gargalo considerável no desenvolvimento. Segue uma imagem de uma seção do *readme.md*:



Entretanto, observou-se erros durante execuções testes e ao ler-se a planilha de instruções fica claro:

JEQ		IMD		IF Z = 1, IMD -> PC
JLT		IMD		IF C = 0, IMD -> PC
JGE		IMD	32b	IF C = 1 OR Z = 1, IMD -> PC

3 Conclusão do Algoritmo

A partir das simples implementações assim, do restante da lógica dentro do Assembly-MIPS32 e do conhecimento do algoritmo BubbleSort, pode-se consumar o código final.

```
.DATA
    ; Define the list numbers, just a few so it runs quickly.
    NUM1 = $0005
    NUM2 = $0009
    NUM3 = $000B

.CODE
    ; Initialize in the registers and then store in RAM.
    LOAD R0 NUM1
    STORE R0 @0000
    LOAD R0 NUM2
    STORE R0 @0001
    LOAD R0 NUM3
    STORE R0 @0002
    LOAD R1 $0003 ; Fixed list length
    ; ------------
    ; Now, implement Bubble Sort

BubbleSort:
    DEC R1 ; Decrement to allow comparison with the element's index.
    LOAD R2 $0000 ; Use R2 for index iteration loop

    ITERATION_LOOP:
        LOAD R3 $0000 ; Use R3 for index comparation loop

        COMPARATION_LOOP:
            ; Check two adjacent elements
            LOAD R4 R3
            INC R3
            LOAD R5 R3
            ; Transform into R6 and R7 for store temporally
            TRAN R4 R6
            TRAN R5 R7
            SUB R5 R4
            ; Note the difference: [j+1] - [j]
            ; Therefore, if [j] is equal to or greater than [j+1], ZF or CF will be 1,
                respectively
            JGE NO_SWAP_CASE
                SWAP_CASE:
                    DEC R3
                    STORE R7 R3 ; Store [j+1] in [j]
                    INC R3
                    STORE R6 R3 ; Store [j] in [j+1]
                NO_SWAP_CASE:
                    ; Nothing happens and it moves on to the next comparison if necessary
                    ; Transform into R6 temporally for subsequent subtraction
                    TRAN R1 R6
                    SUB R6 R3
                    ; Think about the possibles cases
                    ; This took me sometime to realize
                    JEQ AFTER_COMPARATION_LOOP
                    JMP COMPARATION_LOOP
        AFTER_COMPARATION_LOOP:
            INC R2
            TRAN R1 R6
            SUB R6 R2
            JEQ END_EXECUTION
            JMP ITERATION_LOOP

END_EXECUTION:
    ; Sets R0 to 999 to visualize the end
    LOAD R0 $03E7
    JMP END_EXECUTION
```

Foi utilizado como lista inicial os números $\{5, 9, 11\}$, mas a customização é trivial dada modularidade do código.

4 Discussão Final

4.1 Como executar corretamente

1. Setar *RESET* para 1
2. Setar *READ ROM* para 1
3. Setar *READ ROM* para 0
4. Setar *RESET* para 0

Utiliza-se a tecla F9 para alterar o estado do *CLOCK* e, portanto, a execução do programa.

Dependendo do estado de *CLOCK* ao executar o passo 3, algumas células receberão *#REF*, configurando um bug. Para que aconteça a leitura correta, aconselha-se utilizar F9 para setar o *CLOCK* para 1, a fim que ao se realizar o passo 3, o *CLOCK* não execute nada, já que estará na borda de descida.

4.2 Resultados

A tradução foi um sucesso e pôde contribuir com o conhecimento acerca de como computadores são capazes de executar passos tão simples, mas de maneira tão rápida e tão simultânea, permitindo que grandes coisas possam ser possíveis.

Para a tradução correta, foram necessárias modificações na lógica com que os loops e condições são executados, além de que não há uma forma de encerrar o programa em AssemblyExcel16 e utilizou-se um loop infinito.

5 Referências

- Repositório da CPUExcel16: `https://github.com/InkboxSoftware/excelCPU`
- Repositório do trabalho de tradução: `https://github.com/mathdeyvi04/ArqCompExcelCPU`

Também há um vídeo da execução do programa na CPU disponível no repositório de trabalho.